COMP90050 ADBS

Week 10

Q1 Locking

1. The following transactions are issued in a system at the same time. Answer for both scenarios.

   (a) **Scenario 1:** When the value of A is 3, which of the following transactions can run concurrently from the beginning till commit (that is, all operations and locks are compatible to run concurrently with another one) and which ones need to be delayed? Please give an explanation for the delayed transactions. Note that, the order of start of transactions can be deducted from the beginning positions of the transactions in the table given.

   (b) **Scenario 2:** When the value of A is 2, which of the following transactions can run concurrently from the beginning till commit (that is, all operations and locks are compatible to run concurrently with another one) and which ones need to be delayed? Please give an explanation for the delayed transactions. Let's assume T1 starts slightly earlier than others for this case. [1]

T1	T2	T3
Lock (S,A)	Lock (U,A)	Lock (IX,A)
Read A	Read A	Read A
Unlock A	if(A ==3) {	if(A ==3){
	Lock(X,A)	Lock(X,A)
	Write A	Write A
	}	}
	Unlock A	Unlock A

Compatibility Mode of Granular Locks

Current	None	IS	IX	S	SIX	U	X
Request	+I- (Next mode) + granted / - delayed						
IS	+(IS)	+(IS)	+(IX)	+(S)	+(SIX)	-(U)	-(X)
IX	+(IX)	+(IX)	+(IX)	-(S)	-(SIX)	-(U)	-(X)
S	+(S)	+(S)	-(IX)	+(S)	-(SIX)	-(U)	-(X)
SIX	+(SIX)	+(SIX)	-(IX)	-(S)	-(SIX)	-(U)	-(X)
U	+(U)	+(U)	-(IX)	+(U)	-(SIX)	-(U)	-(X)
X	+(X)	-(IS)	-(IX)	-(S)	-(SIX)	-(U)	-(X)

Q1a Locking

	T2	T3
	Lock (U,A)	Lock (IX,A)
T1	Read A	Read A
Lock (S,A)	if(A ==3) {	if(A ==3){
Read A	Lock(X,A)	Lock(X,A)
Unlock A	Write A	Write A
	}	}
	Unlock A	Unlock A

Compatibility Mode of Granular Locks							
Current	None	IS	IX	S	SIX	U	X
Request	+I- (Next mode) + granted / - delayed						
IS	+(IS)	+(IS)	+(IX)	+(S)	+(SIX)	-(U)	-(X)
IX	+(IX)	+(IX)	+(IX)	-(S)	-(SIX)	-(U)	-(X)
S	+(S)	+(S)	-(IX)	+(S)	-(SIX)	-(U)	-(X)
SIX	+(SIX)	+(SIX)	-(IX)	-(S)	-(SIX)	-(U)	-(X)
U	+(U)	+(U)	-(IX)	+(U)	-(SIX)	-(U)	-(X)
X	+(X)	-(IS)	-(IX)	-(S)	-(SIX)	-(U)	-(X)

A == 3: None of the transactions can run concurrently. Only one can run at a time while the other transactions must be delayed

T2 and T3 compatibility (they start first):

- T2's U lock and T3's IX conflict with each other, and the subsequent X locks for A==3 will conflict.

T1 compatibility with either T3/T2:

- T3's IX lock conflicts with T1's S lock.
- T1's S lock and T2's U lock are compatible if T1 gets the lock first, but T1 gets the lock later which does not work.

Q1b Locking

	T2	T3
	Lock (U,A)	Lock (IX,A)
T1	Read A	Read A
Lock (S,A)	if(A ==3) {	if(A ==3){
Read A	A==2 so not run	
Unlock A		
	}	}
	Unlock A	Unlock A

Compatibility Mode of Granular Locks							
Current	None	IS	IX	S	SIX	U	X
Request	+I- (Next mode) + granted / - delayed						
IS	+(IS)	+(IS)	+(IX)	+(S)	+(SIX)	-(U)	-(X)
IX	+(IX)	+(IX)	+(IX)	-(S)	-(SIX)	-(U)	-(X)
S	+(S)	+(S)	-(IX)	+(S)	-(SIX)	-(U)	-(X)
SIX	+(SIX)	+(SIX)	-(IX)	-(S)	-(SIX)	-(U)	-(X)
U	+(U)	+(U)	-(IX)	+(U)	-(SIX)	-(U)	-(X)
X	+(X)	-(IS)	-(IX)	-(S)	-(SIX)	-(U)	-(X)

A == 2: T1 and T2 can run concurrently

T2/T3 compatibility with T1 (which starts first)

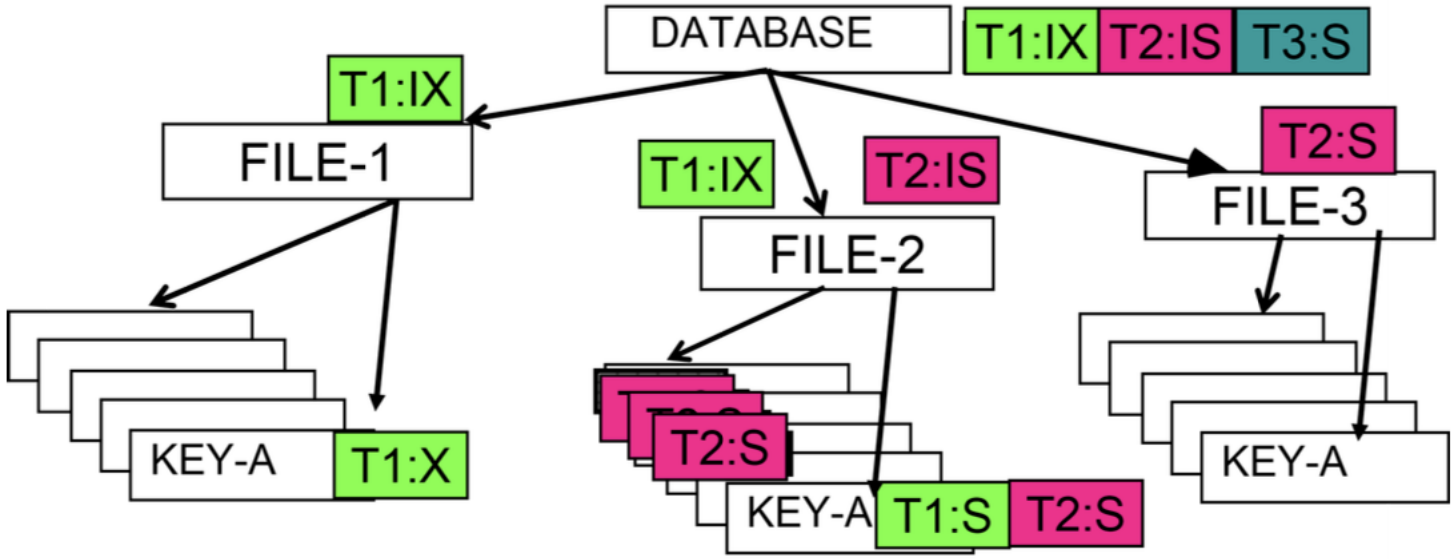
- T2's U lock can be granted if T1 gets the S lock first
- T3's IX lock and T1's S lock conflict with each other

T2 compatibility with T3:

- T2's U lock and T3's IX conflict with each other

Q2 Granular Locks

2. Review the concepts of granular locks then answer the following question. Given the hierarchy of database objects and the corresponding granular locks in the following picture, which transactions can run if the transactions arrive in the order T1-T2-T3? What if the order is T3-T2-T1? Note that locks from the same transaction are in the same colour. We assume that the transactions need to take the locks when they start to run.



Compatibility Mode of Granular Locks

Current	None	IS	IX	S	SIX	U	X
Request	+I- (Next mode) + granted / - delayed						
IS	+(IS)	+(IS)	+(IX)	+(S)	+(SIX)	-(U)	-(X)
IX	+(IX)	+(IX)	+(IX)	-(S)	-(SIX)	-(U)	-(X)
S	+(S)	+(S)	-(IX)	+(S)	-(SIX)	-(U)	-(X)
SIX	+(SIX)	+(SIX)	-(IX)	-(S)	-(SIX)	-(U)	-(X)
U	+(U)	+(U)	-(IX)	+(U)	-(SIX)	-(U)	-(X)
X	+(X)	-(IS)	-(IX)	-(S)	-(SIX)	-(U)	-(X)

T1-T2-T3: T1 and T2 can run while T3 waits

T1's IX lock at the root node is not compatible with T3's S lock at the same node

T3-T2-T1: T3 and T2 can run while T1 waits

T3's S lock at the root node is not compatible with T1's IX lock at the same node

- Order of transactions can impact the set of parallel-running transactions
- Note that granular locks can lead to the delay of transactions at any level in the hierarchy

Q3 Locking Mechanisms

3. With two-phase locking we have already seen a successful strategy that will solve concurrency problems for DBMSs. Then discuss why someone may want to invent something like Optimistic Concurrency control in addition to that locking mechanism.

- *Two-phase locking* (or locking in general) assume the worst: many updates in system and most will lead to conflicts in access to objects
  - Good rationale to pay the overhead of locking to avoid problems

- In DBMS where there aren't many conflicts:
  - E.g. when people tend to work on different parts of data, or most are read operations
  - May be better to allow transactions to run freely and have a simple conflict check when they finish
  - Most times there is no conflict so Increased concurrency with less overhead

- BUT In DBMS where there were really many conflicting writes:
  - *Optimistic concurrency control* means many problems would be observed only after running the transactions
  - A lot of work will be wasted to preserve data consistency

A strategy may be good / bad depending on situation

Lab Sheet

Week 10

FEIT Subject Surveys

- Sent to university email address with the subject "FEIT Subject Surveys”
- Open until Sunday 17 May